

Entregable 02 — Diagramas

Arquitectura de la solución y flujo de decisión del agente

Este documento contiene los dos diagramas que exige el entregable 02: la **arquitectura de la solución** y el **flujo de decisión que usa el agente**. Cada uno va precedido de una descripción que explica qué muestra, cómo leerlo y qué decisión de diseño representa.

El proyecto en una frase

ts-sm es un asistente de voz de seguimiento post-operatorio. Conversa en español colombiano con pacientes recién operados, recorre las seis áreas del seguimiento clínico sin recitarlas como un formulario, responde dudas fundamentado en un corpus de 107 guías clínicas, y decide si el caso debe pasar a un médico humano. Todo el sistema levanta con `docker compose up`.

Los dos diagramas

Diagrama 1 Arquitectura	Las siete capas del sistema, de las superficies web hasta los servicios externos, más el pipeline que preparó el corpus clínico. Muestra dónde vive cada pieza y cuál es el único componente que genera lenguaje.
Diagrama 2 Flujo de decisión	Un turno completo del agente, en once pasos numerados, hasta la decisión de escalar o continuar. Muestra cómo decide y por qué esa decisión no depende de un solo componente.

Cómo verificar que el diagrama corresponde al código

La rúbrica del reto establece que el jurado *“toma elementos del diagrama al azar y los busca en el código”*. Por eso **cada caja lleva impresa su ruta real en el repositorio**, y la última página de este documento es una tabla que mapea cada elemento a su archivo. Ninguna caja de estos diagramas describe algo que no esté implementado.

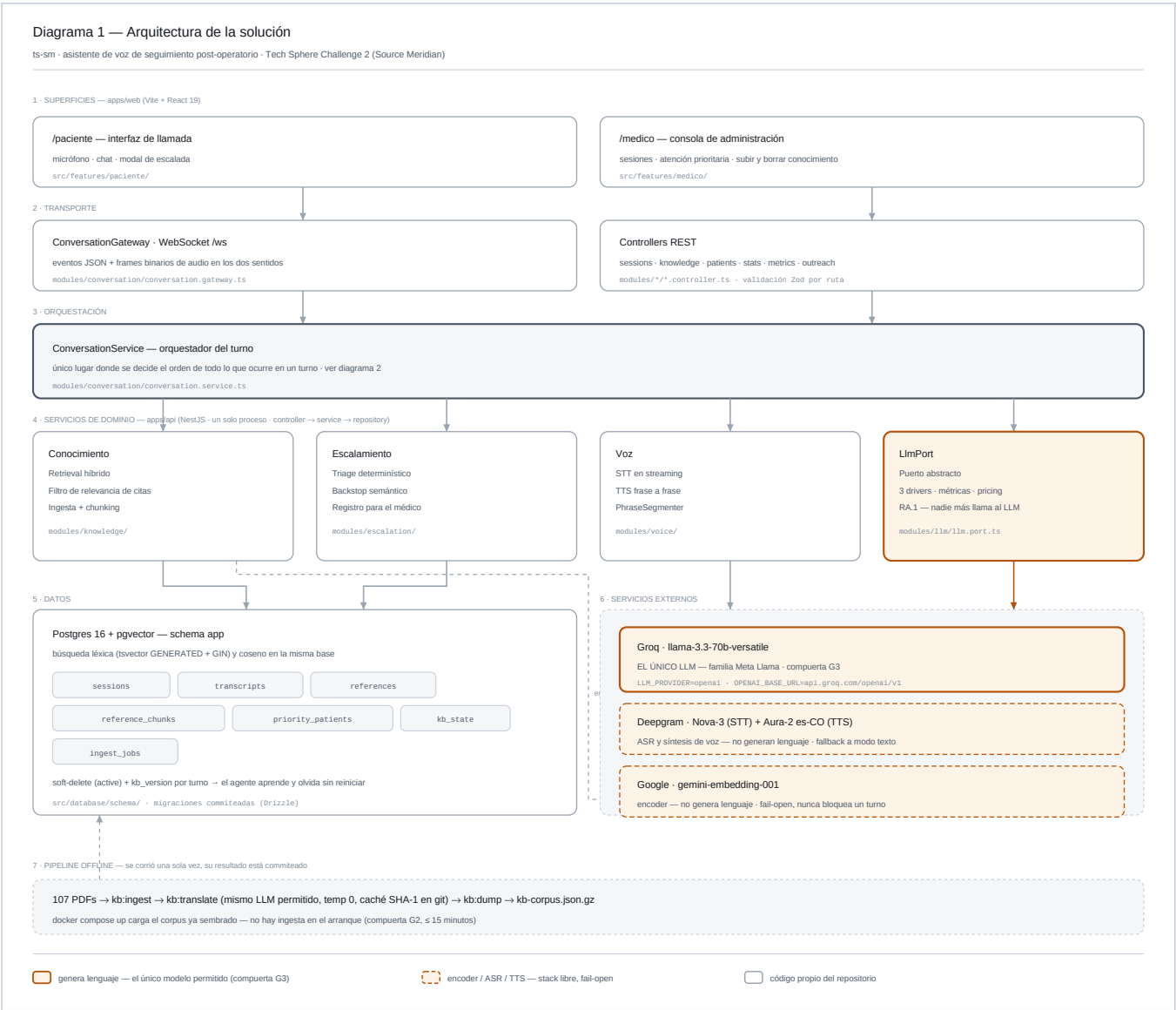
Principio que ordena todo el diseño. En salud, el falso negativo —no alertar cuando había que alertar— es la falla catastrófica. La estructura del diagrama 2 existe por esa asimetría: cuatro caminos independientes hacia la alerta, ninguno hacia atrás.

Arquitectura de la solución

Qué muestra. Siete bandas apiladas. Las capas 1 a 4 son código de este repositorio y siguen el recorrido de una petición: superficie → transporte → orquestación → dominio. La capa 5 es la base de datos, la 6 los servicios externos, y la 7 el pipeline que preparó el corpus clínico una sola vez y cuyo resultado está commiteado.

Por qué un monolito modular. El reto exige que el jurado levante todo en quince minutos o menos en una máquina limpia. Cada servicio adicional es una imagen más que construir y un modo de fallo más el día de la evaluación. Los módulos de NestJS dan las mismas fronteras lógicas sin pagar el costo operativo de la red entre procesos: si esto tuviera que escalar, ya están cortados por donde se cortarían los servicios.

El color codifica el cumplimiento del modelo obligatorio. Hay una sola caja ámbar sólida en todo el sistema: Groq con llama-3.3-70b-versatile. Las demás cajas ámbar van punteadas y son componentes que **no generan lenguaje** —reconocimiento de voz, síntesis y encoders de embeddings—, que el reglamento declara de stack libre. Todo acceso al modelo pasa por LlmPort: los SDK de LLM solo se importan bajo modules/llm/drivers/, y eso se verifica con un grep.



Flujo de decisión del agente

Qué muestra. Un turno completo del agente: desde que llega el mensaje del paciente hasta que el sistema decide escalar el caso a un médico o continuar la conversación. Los once pasos siguen el orden exacto del código, en `ConversationService.handleUserMessageInner()`, y se leen de arriba hacia abajo sin saltos.

Cómo leerlo

Los pasos **1 y 4** son guardas con salida temprana: si la sesión ya está cerrada, o si el paciente confirma el cierre que se le ofreció, el turno termina ahí sin gastar una sola llamada al modelo. El paso **3** persiste el mensaje del paciente *antes* de procesar nada: si todo lo demás falla, lo que dijo el paciente ya está en la base.

Los pasos **5 y 6** son bloques con varias operaciones. El 5 es el triage clínico: normaliza modismos colombianos sobre una copia del texto —el original se guarda intacto para el médico—, extrae magnitudes como “fiebre 38.5” o “dolor 8 sobre 10”, y las fusiona con el estado acumulado de la sesión quedándose siempre con el nivel más severo. El 6 es la recuperación: búsqueda léxica y semántica en paralelo, unión sin fusionar scores, y un filtro de relevancia por embeddings que es el corte que de verdad decide qué ve el modelo.

El paso **8** es el único que corre dos cosas a la vez. El detector de banderas rojas cuesta medio segundo: en serie, ese costo se lo cobraría al paciente en cada turno, incluidos los que no son una emergencia. En paralelo el costo es cero, porque para cuando el modelo termina de responder el detector ya contestó hace rato.

Las cuatro señales — el núcleo del diseño

El paso **9** reúne cuatro señales independientes y el paso 10 las une con un **OR**: cualquiera basta para escalar y **ninguna puede cancelar a otra**. Tres son determinísticas y corren en el servidor. Solo una depende del modelo de lenguaje, y en las pruebas manuales acertaba entre el 40 % y el 50 % de las veces.

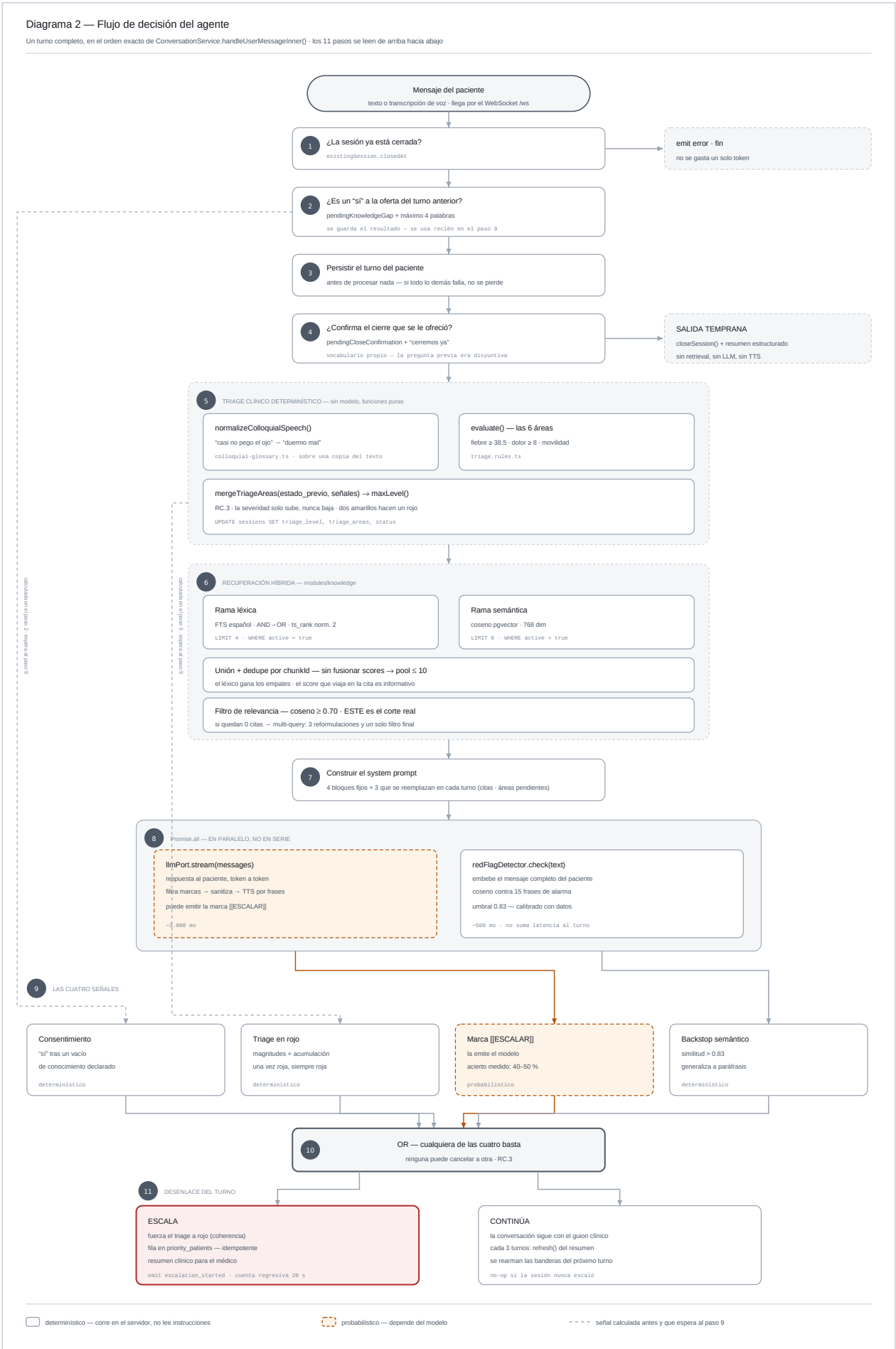
Esa medición es el origen de toda la estructura. El diseño original tenía al modelo como única fuente de verdad para escalar. Al medirlo con el mismo texto de entrada repetido, el resultado cambiaba entre llamadas: “tengo fiebre muy alta y me cuesta respirar” fallaba el cien por ciento de las veces. Dos rondas de ajuste al prompt mejoraron parcialmente, y ahí quedó claro que el problema no se resolvía con más texto de instrucciones. La respuesta fue sacar la decisión de depender de esa fiabilidad, no intentar mejorarla.

Las dos líneas punteadas que bajan por el costado izquierdo no son un adorno: muestran que las señales de los pasos 2 y 5 se calculan mucho antes y esperan al paso 9. Solo las señales del modelo y del detector semántico se resuelven dentro del bloque paralelo. Ninguna de las cuatro hace corto-circuito: las cuatro se evalúan completas, siempre, y las cuatro dejan rastro en los logs.

La consecuencia que importa. Un atacante que convenza al modelo de no escalar —o un turno en el que el modelo simplemente falle— sigue teniendo enfrente el triage determinístico y el detector semántico, que no leen instrucciones: solo miden números y similitud. Si Groq se cae, si el servicio de embeddings se cae, el sistema sigue pudiendo alertar a un médico.

Diagrama 2 — Flujo de decisión del agente

Un turno completo · once pasos en el orden exacto de handleUserMessageInner()



Trazabilidad — dónde está cada caja en el código

Cada elemento de los dos diagramas corresponde a un archivo del repositorio. Rutas relativas a apps/api/src/ salvo donde se indique.

Elemento del diagrama	Archivo
/paciente · /medico	apps/web/src/features/{paciente,medico}/
ConversationGateway · WebSocket /ws	modules/conversation/conversation.gateway.ts
ConversationService — orquestador	modules/conversation/conversation.service.ts
System prompt y sus 7 bloques	modules/conversation/conversation.prompt.ts
Marca [[ESCALAR]] y su filtro	modules/conversation/text-sanitizer.ts
LlmPort — el puerto abstracto	modules/llm/llm.port.ts
Los 3 drivers de LLM	modules/llm/drivers/
Rama léxica · rama semántica · unión	modules/knowledge/retrieval.service.ts
Filtro de relevancia (coseno ≥ 0.70)	modules/knowledge/citation-relevance.service.ts
Multi-query (3 reformulaciones)	conversation.service.ts → getQueryVariations()
Ingesta · chunking · idioma	modules/knowledge/{ingestion.service, chunker, detect-language}.ts
normalizeColloquialSpeech()	modules/escalation/colloquial-glossary.ts
evaluate() · mergeTriageAreas() · maxLevel()	modules/escalation/triage.rules.ts
Backstop semántico (umbral 0.83)	modules/escalation/red-flag-detector.service.ts
Las 15 frases de alarma	modules/escalation/red-flag-phrases.ts
Registro para el médico	modules/escalation/escalation.service.ts
STT · TTS · PhraseSegmenter	modules/voice/voice.service.ts
Cliente de embeddings (Gemini)	modules/embeddings/embedding.client.ts
Métricas por turno	modules/metrics/turn-metrics.ts
Tablas del schema app	database/schema/
Pipeline offline del corpus	database/kb-{ingest, translate, dump}.ts

Los archivos fuente de los diagramas están en docs/diagramas/ como SVG vectorial autocontenido.